CPT102 Data Structures: Lecture 4: Recursion

Thomas Selig

XJTLU

March 23, 2026

Reminders/announcements

- Week 3 practice exercise sheet on Learning Mall.
  - Solution later this week.
- New office hours!
  - Monday 14:00-16:00 (not around this week, sorry);
  - Tuesday 13:00-14:00 (this afternoon I'll be in SC331, welcome to stop by any time);
  - Friday 13:00-14:00.
- CW introduction later this week.

Feedback

Key message

Feedback, in any (reasonably respectful) form, is always welcome!

- I have changed last week's pseudocode to more Java-adjacent style.
- I will try (!) to speak more slowly.
- Please continue pointing out typos or errors in slides.
  - Slides are new this year, inevitable there will be issues.
  - Rewards (chocolate/candies) for spotting errors.

Important reminder!

The slides for this course do not contain all the information. I provide a lot of additional information on the whiteboard during the lectures.

- Attend the lectures, and pay attention! 

Contents of today's lecture

All about recursion

about recursion

about recursion

about recursion

about recursion

- Introducing the concept of recursive algorithms.
- Complexity analysis of (reasonably) basic recursive algorithms.
- Comparison: recursive vs iterative solutions.

Lecture Learning Outcomes

By the end of today's lecture, you should be able to:

A. Utilise recursive techniques to solve specified problems.

B. Analyse the complexity of simple recursive algorithms.

C. Compare recursive and iterative solutions to a given problem, assessing their relative strengths and weaknesses.

Warm-up: factorial

For a non-negative integer n, we define the *factorial* of n, denoted $n!$ (read "n factorial") by:

$$\boxed{n! = n \times (n-1) \times \cdots \times 2 \times 1}$$

Algorithm 1: calculate the factorial

Procedure FACTORIAL(n)

```
fact ← 1;
for i ← 2 to n do
    fact ← fact * i;
return fact;
```

Question

Time and space complexities?

Recursive version

Previous procedure is fine, but:

- Need to be careful on bounds of loop;
- Doesn't quite capture *essence* of factorial, which is given by the recurrence relation:

$$\boxed{n! = n \times (n-1)!}$$

This gives:

Algorithm 2: Recursive factorial

Procedure FACTORIAL(n)
if $n \leq 1$ **then**
  return 1;
return $n *$ FACTORIAL($n-1$);

Recursive algorithms: what? how?

Definition

A **recursive algorithm** is an algorithm that calls itself.

- Usually, recursive call is on input of smaller size.
- In order to terminate, a recursive algorithm must include:
  1. A **base case**, or **stopping rule**: this indicates when the algorithm terminates;
  2. A **recursive step**, where the algorithm calls itself (they key ingredient!).
- Successive recursive calls are stored in memory on the **call stack**.
  - Space complexity = sum of space complexities of each recursive call.

Recursive step: how does it work?

The recursive step is the key ingredient in a recursive algorithm. It can be thought of as follows.

- You are given an *instance* of a particular problem ($\equiv$ the input of the algorithm) to solve.
  - Given *this integer*, what is its factorial?
  - Given *this list* and *this value*, what is the value's index of first occurrence in the list?
- Imagine that there is a "Recursion God", who can tell you how to solve *any smaller instance* of the problem ($\equiv$ perform the algorithm on any input of smaller size).
  - Factorial of *any integer* smaller than *this integer*.
  - Index of first occurrence of *this value* for *any sub-list* of *this list*.
- The recursive step solves the given *instance* by asking the Recusion God for solution on one or more *smaller instances*.

Recursive step and base case

- Recursive step is found by answering question: If I can solve this problem for *smaller instances*, how could I use those solutions to solve *this instance*?
- The base case usually appears when the recursive step breaks down or stops working.
  - That is, when there's no such thing as a *smaller instance* of the problem.

Tip

Often, it's easier to start with the recursive step. The base case appears naturally when asking: "when does this no longer work"?

Example 1: factorial

- Recursive step? Easy: if Recursion God can give me $\textsc{Factorial}(n-1)$, then I can calculate $\textsc{Factorial}(n)$ $(= n \times \textsc{Factorial}(n-1)\ )$
- Base case: when does this stop working? When $n = 0$, since $0! = 1 \neq 0 \times (-1)!$
  - Actually, can stop when $n = 1$, as in Algorithm 2.

Algorithm 2: Recursive factorial

Procedure $\textsc{Factorial}(n)$

if $n \leq 1$ **then**

  return 1; ▷ base case

return $n * \textsc{Factorial}(n-1)$; ▷ recursive step

Recursive factorial: space complexity

- How does it work?

$$
\begin{aligned}
5! &= 5 \times 4! \\
4! &= 4 \times 3! \\
3! &= 3 \times 2! \\
2! &= 2 \times 1! \\
1! &= 1
\end{aligned}
$$

- Space complexity analysis:
  - Each line above represents a recursive call; each recursive call is stored on the **call stack**.
  - Here: each recursive call requires constant memory space (it returns an integer), and there are $\mathrm{O}(n)$ recursive calls.

  Conclusion: space complexity $= \mathrm{O}(n)$.
- How does this compare to the iterative version (Algorithm 1).

Recursive factorial: time complexity

Question

What is the time complexity of the recursive factorial algorithm?

- Each call to FACTORIAL requires:
  - One check ($n \leq 1$);
  - One multiplication.
- This gives us: $T(n) = 2 + T(n-1)$.
- Arithmetic progression! Easy to show: $T(n) = 2n + T(0) = \mathrm{O}(n)$.
  - **Linear** time complexity. How does this compare to iterative version?
- In fact, we will see that when $T(n) = \mathrm{O}(n^a) + T(n-1))$, then $T(n) = \mathrm{O}(n^{a+1})$ (here, $a = 0$).

Example 2: Fibonacci sequence

- The famous Fibonacci sequence is given by:
  - Fib(0) = 0, Fib(1) = 1;
  - Fib(n) = Fib(n-1) + Fib(n-2) if n >1.
- Easy enough to implement:

Algorithm 3: naive Fibonacci implementation

Procedure NaiveFib(n)
if $n \leq 1$ **then**
  return n;
return NaiveFib($n-1$) + NaiveFib($n-2$);

- However...

```
run:
Fib(50) = 12586269025
BUILD SUCCESSFUL (total time: 38 seconds)
```

~40 seconds to run on n=50!!!

Why so slow?

Fib(50) = Fib(49) + Fib(48)

Fib(49) = Fib(48) + **Fib(47)**

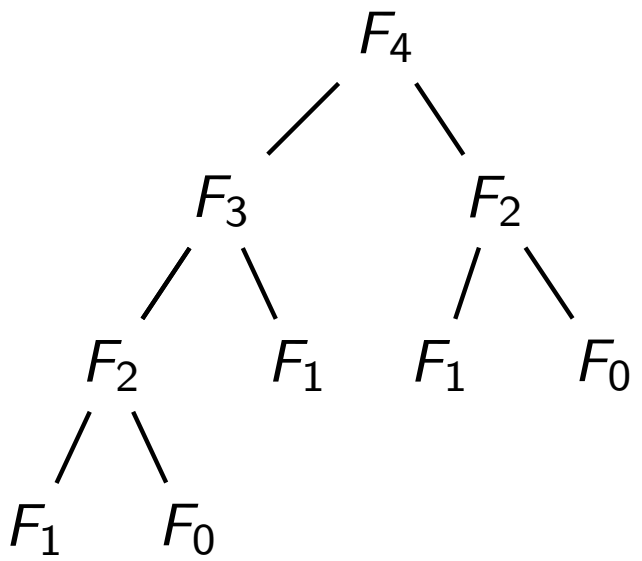
Fib(48) = **Fib(47)** + Fib(46)

Fib(48) = **Fib(47)** + Fib(46)

- Observations.
  - Fib(50) and Fib(49) calculated (called) once.
  - Fib(48) calculated $1 + 1 = 2$ times.
  - **Fib(47)** calculated $1 + 2 = \mathbf{3}$ times.
- In general, need to calculate Fib(n) every time we calculate Fib(n+1) and every time we calculate Fib(n+2).
- This is the Fibonacci sequence itself! Fib(1) will need to be called Fib(50) $\approx$ 20 billion times!
- This implies that the time complexity $T(n)$ of NaiveFib is at least Fib(n).

Another representation: recursion tree

- We can also represent the recursive calls of a recursive algorithm through its **recursion tree**.
- If we write F_k for each call to NaiveFib(k), the following shows the recursion tree for NaiveFib(4).



If we write $a_k(n)$ for the number of calls to NaiveFib(k) in the execution of NaiveFib(n), we see that:

- $a_k(n) = \mathrm{Fib}(n+1-k)$ for $k \in [1, n]$
- In particular: $\boxed{a_1(n) = \mathrm{Fib}(\mathrm{n})}$

Formal analysis: time complexity

Algorithm 3: naive Fibonacci implementation

Procedure NaiveFib(n)
if $n \leq 1$ **then**
  return n;
return NaiveFib($n-1$) + NaiveFib($n-2$);

- Clearly, we have: $T(n) = T(n-1) + T(n-2) + c$
  - The constant c encompasses check ($n \leq 1$), addition, return, etc.
- Can solve explicitly; we get:

$$\boxed{T(n) \sim C \cdot \phi^n}$$

where $\phi := \dfrac{1+\sqrt{5}}{2} \approx 1.618\ldots$ is the *golden ratio*.

- Therefore, **exponential** complexity: SLOW!

Digression: complexity scales

Scale name	Big-Oh	Effect of $n \to n * 10$
Logarithmic	$\mathrm{O}(\log(n))$	$T(n) \to T(n) + c$
Linear	$\mathrm{O}(n)$	$T(n) \to 10 * T(n)$
Log-linear	$\mathrm{O}(n \cdot \log(n))$	$T(n) \to 10 * (T(n) + c \cdot n)$
Quadratic	$\mathrm{O}(n^2)$	$T(n) \to 100 * T(n)$
Cubic	$\mathrm{O}(n^3)$	$T(n) \to 1000 * T(n)$

- Generally, for large n, don't want much worse than quadratic.
- For exponential $\mathrm{O}(2^n)$, when $n \to n+10$, we have: $T(n) \to 2^{10} * T(n) \approx 1000 * T(n)$.
  - In practical terms, exponential algorithms are impossible to run, even for medium-sized inputs!

Improved Fibonacci: FibList

Let's return to our Fibonacci problem.

- The issue was **repeated calculations**: Fib(n) calculated in Fib(n+2) **and** Fib(n+1).
  - We should only need to calculate it once, then "remember" the value.
- Idea: use a list!

Algorithm 4: calculating first n terms of Fibonacci sequence

Procedure FibList(n)

if $n = 0$ **then**
  return [0];

if $n = 1$ **then**
  return [0, 1];

`list` $\leftarrow$ FibList($n-1$);

`nextTerm` $\leftarrow$ `list.get`($n-1$) + `list.get`($n-2$);

`list.add(nextTerm);`

return `list`;

Improved Fibonacci: FastFib

Using this, calculating Fib(n) is easy!

Algorithm 5: a fast Fibonacci algorithm

Procedure FastFib(n)
`fibList` $\leftarrow$ FibList(n);
return `fibList.get`(n);

- Notice that the FastFib procedure is not itself recursive.
- Instead, the recursive step is in an auxiliary sub-procedure FibList.
  - We refer to FibList as a *helper* procedure / algorithm.
- The use of helper algorithms to solve problems recursively is a common strategy to optimise their time and space complexities.
  - More on this shortly.
- The complexity of FastFib is just the complexity of FibList, so let's analyse that.

Improved Fibonacci: complexity analysis

Question

What are the time and space complexities of FibList?

- **Time complexity**. Here we have: $T(n) = T(n-1) + c$.
  - Same as recursive factorial!

  Therefore, time complexity is **linear** $\mathrm{O}(n)$. Much (much) faster!
- **Space complexity**:
  - We have n recursive calls;
  - Call to FibList(k) returns an array of length $k+1$.
  - Therefore, in total, we get $M(n) \sim \sum_{k=1}^{n} k = \mathrm{O}(n^2)$.

  Quadratic space complexity.

New question

Can we do better (in space)?

Yet another Fibonacci

Key idea

Storing all values Fib(0), ..., Fib(n) is unnecessary. In fact, we only need Fib(n) and Fib(n-1).

Algorithm 6: space- and time-efficient recursive Fibonacci

Procedure LastTwoFib(n)
Output: The last two values Fib(n) and Fib($n-1$) in the Fibonacci sequence
if $n = 1$ **then**
    return [0, 1];
`prev` $\leftarrow$ LastTwoFib($n-1$);
`nextTerm` $\leftarrow$ `prev`[0] + `prev`[1];
return [`prev`[1], `nextTerm`];

Procedure EfficientFib(n)
if $n = 0$ **then**
    return 0;
return LastTwoFib(n)[1];

EfficientFib: complexity analysis

Relatively straightforward analysis:

- **Time complexity**. Same as previously: $T(n) = T(n-1) + c$, so **linear** time complexity.
- **Space complexity**:
  - We still have n recursive calls, i.e. n items on the call stack.
  - But this time each recursive call returns an array of length 2, i.e. requires only constant memory space.

  Therefore, we get **linear** space complexity.

Question

Can we do better still?

- Not if we want to use recursion.
- Intuitively, if we need n recursive calls, we will need at least $\mathrm{O}(n)$ memory space.

IterFib: an iterative Fibonacci algorithm

Algorithm 7: an iterative Fibonacci algorithm

Procedure IterFib(n)

if $n = 0$ **then**

    return 0;

previous $\leftarrow$ 0, current $\leftarrow$ 1;

for $i \leftarrow 2$ **to** n **do**

    temp $\leftarrow$ current;

    current $\leftarrow$ previous + current;

    previous $\leftarrow$ temp;

return current;

- **Time complexity**: linear $\mathrm{O}(n)$ (one loop with constant operations);
- **Space complexity**: constant $\mathrm{O}(1)$.

Iterative vs recursive: pros and cons

- Every recursive algorithm can be transposed into an iterative version.
  - In fact, **any computer program** can be expressed using just a **sequence of conditional statements and loops** (C. Böhm and G. Jacopini, *Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules*, 1966).
- The converse is also (sort-of) true.
  - Some languages, especially in **functional** programming, only allow recursion (no loops!).
- Recursive algorithms:
  - Are generally more elegant, can be expressed more compactly, capture the "essence" of the problem well;
  - Can be slower if expressed naively, generally require more memory space due to the call stack (risk of overflows).
- Iterative algorithms:
  - Are usually more efficient in space and sometimes faster;
  - Can be more complex to express.

Another example: linear search

Recall from previous weeks:

Algorithm 8: iterative linear search

Procedure LinearSearch(`arr`, `val`)

```
foreach item in arr do
    if item = val then
        return true;
return false;
```

- Can we write a recursive version?
  - Yes, by previous slide!
- Natural idea: if `arr[0] = val`, we've found the value; otherwise, search in the remainder of the array.
- However, don't want to do any array-copying, so need to be cautious when defining "remainder of the array".
- Solution: use a helper procedure!

Recursive linear search

Algorithm 9: recursive linear search

Procedure LinearSearchHelper(`arr`, `val`, `startIndex`)
Output: **true** if `arr` contains `val` at an index $\geq$ `startIndex`, **false** otherwise
if *startIndex = arr.length* **then**
    return false; ▷ base case
if *arr[startIndex] = val* **then**
    return true;
return LinearSearchHelper(`arr`, `val`, `startIndex` + 1);

Procedure LinearSearch(`arr`, `val`)
return LinearSearchHelper(`arr`, `val`, 0);

- Last two returns can be combined using an **or** statement.
- **Time complexity**: Since `startIndex` ranges from 0 to n through the recursive calls, and each call has a constant number of operations, the time complexity is **linear** $\mathrm{O}(n)$ (same as iterative).
- **Space complexity**: There are n recursive calls, each occupying constant memory space, so the space complexity is **linear** $\mathrm{O}(n)$.

Recursive complexity analysis: decrease-and-conquer

- So far, we have encountered a number of recursive algorithms where the time complexity satisfies a recurrence of the form:

$$\boxed{T(n) = T(n-1) + \mathrm{O}(1)}$$

  Such algorithms have **linear** complexity $\mathrm{O}(n)$.
- More generally, if a recursive algorithm satisfies:

$$\boxed{T(n) = T(n-1) + \mathrm{O}\left(n^k\right)}$$

  then we get $T(n) = \mathrm{O}\left(n^{k+1}\right)$.
- Such algorithms with recurrences of the form

$$\boxed{T(n) = 1 \cdot T(\alpha_n) + f(n)}$$

  for some function $f(n)$ and where $\alpha_n < n$, are called **decrease-and-conquer**.
  - If $\alpha_n = n - k$ for some fixed k (e.g., $k = 1$, as above), we say they *decrease by a constant*.

Recursive complexity analysis: linear recurrences

- Decrease-and-conquer: $\boxed{T(n) = 1 \cdot T(\alpha_n) + f(n)}$

  Intuitively, only need to solve one smaller problem to solve original problem.
  - The α_n is the size of the sub-problem.
  - The $f(n)$ is the additional cost of assembling the solution to the original problem.

  As long as $f(n)$ is not too large, such solutions are time-efficient.

- More costly solutions:
  - Naive Fibonacci: $\boxed{T(n) = T(n-1) + T(n-2) + c}$
  - Tower of Hanoi (exercise sheet), the ruler method (tutorial) both give: $\boxed{T(n) = 2 \cdot T(n-1) + c}$

  These all have **exponential** time complexity. SLOW!

Towards divide-and-conquer techniques

Intuition from previous slide

- If only need to solve a *single* smaller problem, then solutions are time-efficient (at worst polynomial) no matter size of smaller problem.
- If need to solve *multiple* smaller problems, all of size at least $n - k$ (fixed k), then we get exponential time complexity.

- **Consequence**: To get efficient solutions in second case, need sub-problems to be "much smaller" than original problem.
- What does "much smaller" mean?
  - Size of sub-problems is (at most) n/b (for some $b > 1$), rather than $n - k$.
  - Reduce by multiplicative, not additive, constant.

Divide-and-conquer: what?

- A **divide-and-conquer** algorithm satisfies a recurrence relation of the form: $\boxed{T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)}$
  - a = number of sub-problems to solve ($a > 1$);
  - b = reduction factor: sub-problems have size $\frac{n}{b}$ ($b > 1$);
  - $f(n)$ = additional cost of re-assembling solution to the original problem from solutions to sub-problems.
- In Week 6 we will see the **Master Theorem** that gives a direct formula to estimate $T(n)$, depending on relative values of a, b, $f(n)$.
- Example: If $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + n$, then we get **log-linear** complexity $T(n) = \mathrm{O}\left(n \cdot \log(n)\right)$.
  - We will see several sorting algorithms with this behaviour (Week 6).
- N.B.: if $a = 1$, we are in the *decrease*-and-conquer case, and say that the algorithm *decreases by a constant factor*.
  - Example: binary search (next week).

Divide-and-conquer: why?

Assume that $a = b$.

- Break problem down into a problems, all of equal size.

Decrease-and-conquer

$$T(n) = T(n-1) + f(n)$$

- Cost of re-assembly $= f(n)$.
- Number of recursive calls needed $= n$.
- Intuitively: $T(n) \approx \mathrm{O}\left(n \cdot f(n)\right)$.

Divide-and-conquer

$$T(n) = a \cdot T(n/a) + f(n)$$

- Cost of re-assembly $= f(n)$.
- Number of recursive calls needed $\approx \log(n)$.
- Intuitively: $T(n) \approx \mathrm{O}\left(\log(n) \cdot f(n)\right)$.

Intuition

Divide-and-conquer algorithms are much more efficient than decrease-and-conquer ones, since $\log(n) \ll n$.

Summary

- Concept of recursive algorithms.
- Some classical examples.
  - Complexity analysis.
  - Optimisation where naive implementations were inefficient.
- Recursive vs iterative: pros and cons.
- General techniques: decrease-and-conquer, divide-and-conquer, linear recurrence recursions.
  - Complexity comparison.